Funciones y Parámetros

En JavaScript, a pesar de ser un lenguaje orientado a objetos, no es necesario crear objetos para implementar determinadas funcionalidades. Cuando tengamos que crear funcionalidades que se van a repetir o, simplemente, dividir el código en unidades funcionales más simples para una mejor comprensión y depuración podemos emplear las **funciones**.

Una **función** no es más que una serie de **instrucciones con nombre**, a las que, opcionalmente, se les **puede proporcionar datos** para modificar su comportamiento y que, también opcionalmente, **pueden retornar un valor**. Las funciones pueden ser invocadas **cuando se produce una situación** determinada **(evento), dentro de un script** o, incluso, **dentro de otra función**.

Hay tres formas de definir funciones:

- **Function declaration**: Su sintaxis es:

```
function nombreFuncion([lista_parametros]){
   instrucciones;
   [return expresión;]
}
```

La **declaración** de funciones **crea un objeto con el nombre de la función**

Ver código en JSFiddle

NOTA: Si **queremos emplear una función definida en un script** desde **otro script** o desde código **embebido** podremos hacerlo **siempre y cuando**

hayamos cargado el script antes de invocar la función.

Si en **index.js** tenemos:

```
function f1(){
    console.log("En F1");
}
```

Y en **index.html**:

```
<script src="index.js"></script>
<script>
    f1();
</script>
```

La salida sería

```
En F1
```

Si en **index.html** ponemos:

```
<script>
   f1();
</script>
<script src="index.js"></script>
```

La salida:

```
Uncaught ReferenceError: f1 is not defined
```

- **Function expression**: Se declara la función asignándola directamente a una variable:

```
let funcion = function([listaParámetros]) { …}
```

- **Funciones Flecha (ES6)** : Es una forma **compacta** de definir funciones:

```
(lista_parámetros) => expresión;
```

  Se retorna el resultado de evaluar la expresión

- La **expresión puede incluir múltiples instrucciones** en cuyo caso deben ir **entre llaves** y, si queremos retornar un valor emplearemos `return`.
- **Si sólo hay un parámetro** se pueden **omitir los paréntesis**
- **Si no hay ningún parámetro** hay que **indicar los paréntesis**

Ver código en JSFiddle

Las funciones de flecha tienen una característica que las hace muy interesantes en ciertos momentos y que comentaremos más adelante.

NOTA: Realmente las funciones son **objetos** como veremos en la siguiente unidad.

Parámetros

Para modificar el comportamiento de una función se le puede proporcionar información en forma de **parámetros**. Un parámetro es una variable local a la función en la que se almacena la información proporcionada para ejecutar la función.

En el caso de JavaScript los parámetros son una lista de variables con nombre. El paso de parámetros es **posicional**, esto es, el primer valor pasado por el usuario irá al primer parámetro, el segundo al segundo y así sucesivamente.

Los parámetros **asumirán el tipo de datos de la expresión que se les asigne**.

Los parámetros pueden recibir valores por defecto y, si no se pasa su valor, asumen como valor **undefined.**

NOTA: En una función (que recordemos que es un objeto) podemos emplear la propiedad **arguments** para acceder a una matriz de argumentos y **lenght** para saber cuántos argumentos hemos recibido. De hecho, a una función JavaScript se le pueden pasar tantos valores como queramos (independientemente del número de parámetros que hayamos definido):

Ver código en JSFiddle

Número variable de parámetros (ES6)

En las versiones actuales de Javascript podemos indicar un número variable de argumentos mediante la expresión `…argumentos` **(rest operator)**

En dicho parámetro se recibirá una **matriz** (veremos matrices más adelante) con los argumentos recibidos. Lógicamente, este parámetro especial **sólo puede aparecer una vez y**

al final de la lista de parámetros:

Ver código en JSFiddle

NOTA: Con el símbolo . . . tenemos dos operadores:

- `rest` : Se usa para convertir un conjunto de parámetros en una matriz
- `spread` : Lo contrario. Desde una matriz retorna un conjunto de elementos (el opuesto al anterior)

Expresiones e instrucciones y su relación con funciones

Las expresiones de funciones **no pueden ser empleadas sin asignarlas a una variable o pasarlas a un método**.

```
function() {

    console.log("Hola");

}
```

```
Uncaught SyntaxError: Function statements require a function name
```

Funciones callback

Un uso muy importante de las funciones y que proviene de su característica de ser objetos es que **pueden ser pasadas como argumentos a otras funciones** para que sean llamadas **(callback)** para ser ejecutadas dentro de la primera funcion.

NOTA: Lo que tenemos que pasar es el **objeto función** propiamente dicho, **no la llamada a la función**:

```
function ejecuta(f1,a,b){
    f1(a,b);
}

// Declaración de Función
function saludo(nombre){
    document.write(`Hola ${nombre}<br>`);
}

ejecuta(saludo,'Juan');

// Function Expression
ejecuta(function() {
    document.write('Adiós<br>');
});

// Función Lambda
ejecuta((dato1,dato2) => document.write(dato1+dato2) ,10,20);
```

Este seria el resultado del Script anterior

```
Hola Juan
Adiós
30
```